LangChain

LangChain is a framework that simplifies the development of applications using LLMs. It provides reusable components to connect LLMs with prompts, tools, databases, APIs and external data sources.

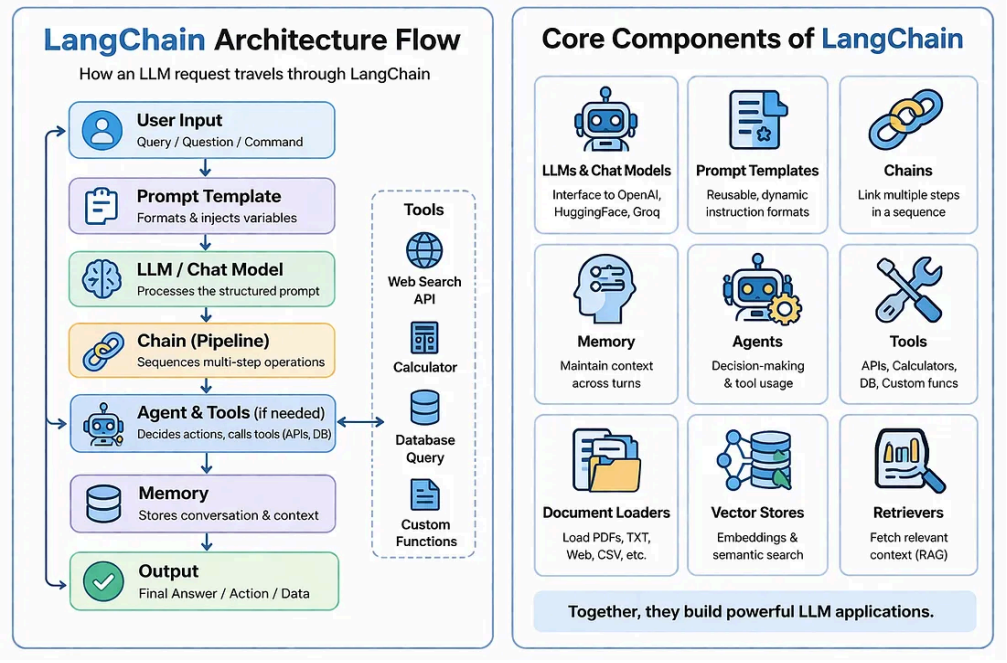
Key features:

- Model Integration: Connects with models such as OpenAI, Gemini, Claude and open-source LLMs.
- Prompt Templates: Create reusable and structured prompts.
- Chains: Connect multiple operations into a sequence.
- Tools: Allow LLMs to interact with external APIs, search engines and databases.
- Retrieval (RAG): Connect LLMs with external knowledge sources.
- Memory: Maintain conversational context.

Example:

An AI customer support application:

1. The user asks a question.
2. LangChain processes the prompt.
3. It retrieves relevant information from a knowledge base.
4. The LLM generates an answer.
5. The answer is returned to the user.



LangGraph

LangGraph is a framework built on top of the LangChain ecosystem for creating stateful, multi-step agent workflows.

Unlike a simple sequential chain, LangGraph allows workflows to have:

- Cycles and loops.
- Conditional branching.
- Multiple agents.
- Shared state.
- Human-in-the-loop approval.
- Checkpointing and persistence.

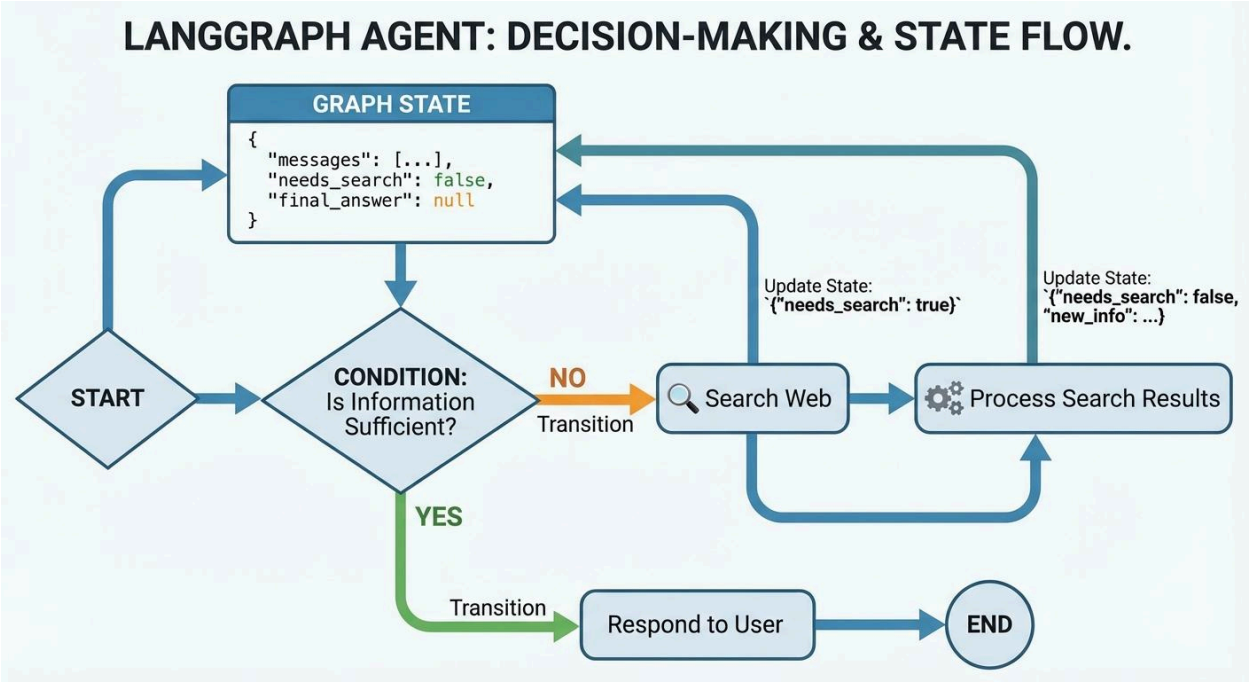
Important concepts:

Component	Description
Nodes	Individual tasks or agents
Edges	Connections between nodes
State	Shared information maintained throughout execution
Conditional Edges	Decide which node executes next
Checkpointing	Saves workflow state
Graph	Complete workflow structure

Example: AI Research Agent

1. The Research Agent collects information.
2. Analysis Agent analyses the information.
3. Conditional check determines whether more research is needed.

4. If more research is needed, it loops back.
5. If sufficient, the Report Agent generates the final report.



LangChain vs LangGraph

Feature	LangChain	LangGraph
Primary purpose	Building LLM applications	Building complex agent workflows
Structure	Chains and components	Graph-based architecture
Execution	Often sequential	Sequential, parallel and cyclic
State management	Available through integrations	Central state management

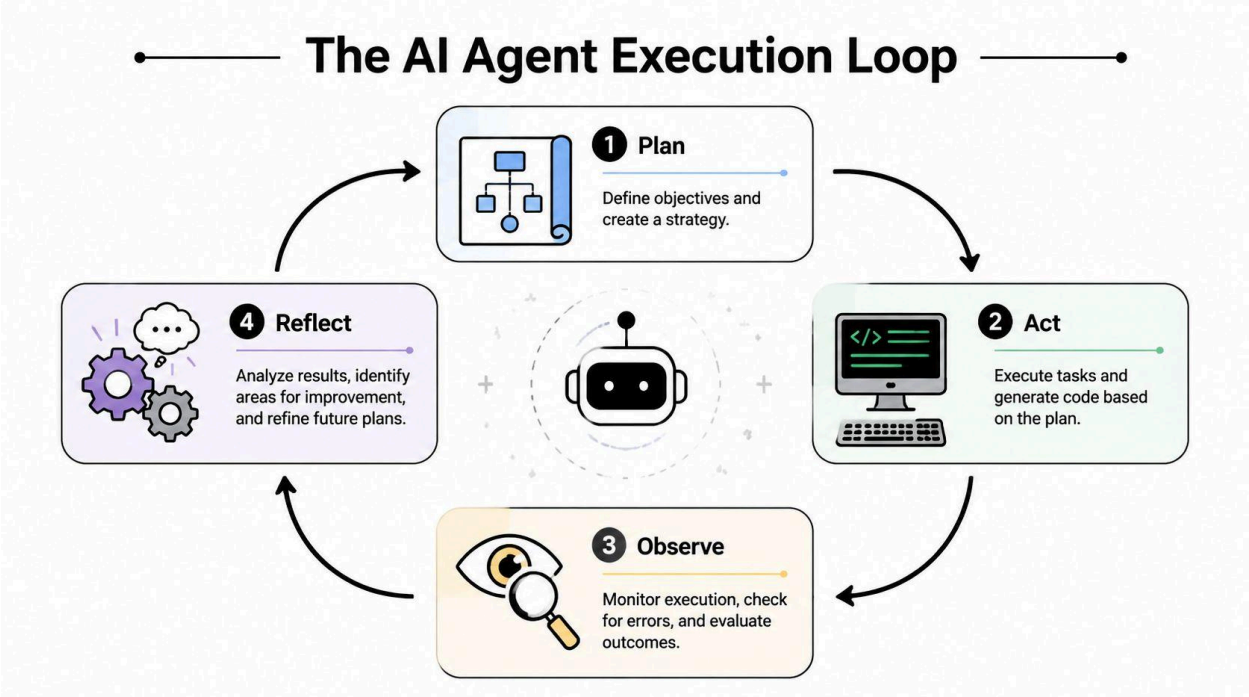
Conditional routing	Supported	Built into graph workflows
Multi-agent systems	Supported	Particularly suitable
Human intervention	Supported	Supports interrupt and resume workflows

Agent Lifecycle

An AI Agent Lifecycle describes the different stages an AI agent goes through, from receiving a task to completing it.

An AI agent does more than generate responses. It can understand instructions, plan actions, execute tools, observe results and modify its approach.

Stages of an Agent Lifecycle



1. Perception / Input

- The agent receives input from the user or an external system.
- It identifies the user's intention.
- It extracts relevant information from the input.

Example: "Find me two laptops under ₹1,20,000."

2. Planning

- The agent breaks down the task into smaller steps.
- Determines which tools are required.
- Decides the order of execution.

Example:

- Search for laptops.
- Filter by price.
- Compare specifications.
- Recommend the best two options.

3. Reasoning and Decision-Making

- The agent analyses available information.
- Evaluates possible actions.
- Selects the most appropriate next step.

4. Action / Execution

- The agent executes its planned actions.
- Calls APIs, databases or external tools.
- Retrieves information or performs operations.

Example: Calling a product-search API.

5. Observation

- The agent observes the result of its actions.
- Checks whether the tool returned useful information.
- Identifies errors or missing information.

6. Evaluation and Reflection

- Evaluates whether the task has been completed successfully.
- Identifies mistakes.

- Revises its plan if required.

7. Response / Completion

- Produces the final response.
- Returns the result to the user.
- Ends the execution when the objective is achieved.

Agent Lifecycle Example

User: "Book a hotel in Bengaluru under ₹5,000 per night."

Stage	Agent activity
Input	Understands the hotel booking request
Planning	Identifies location, budget and dates
Reasoning	Decides to search hotel platforms
Action	Calls hotel-search API
Observation	Receives available hotel details
Evaluation	Checks whether options satisfy the budget
Completion	Presents suitable hotels or proceeds with booking after approval

The lifecycle is not always linear. Agents can repeat planning, execution and observation until the objective is achieved.

Multi-Agent Coordination Patterns

A Multi-Agent System (MAS) consists of multiple AI agents that collaborate to complete a task.

Instead of assigning all responsibilities to one agent, a complex task is divided among specialised agents.

Each agent has:

- A specific role.
- Its own instructions.
- Access to relevant tools.
- A defined responsibility.
- The ability to communicate or share information with other agents.

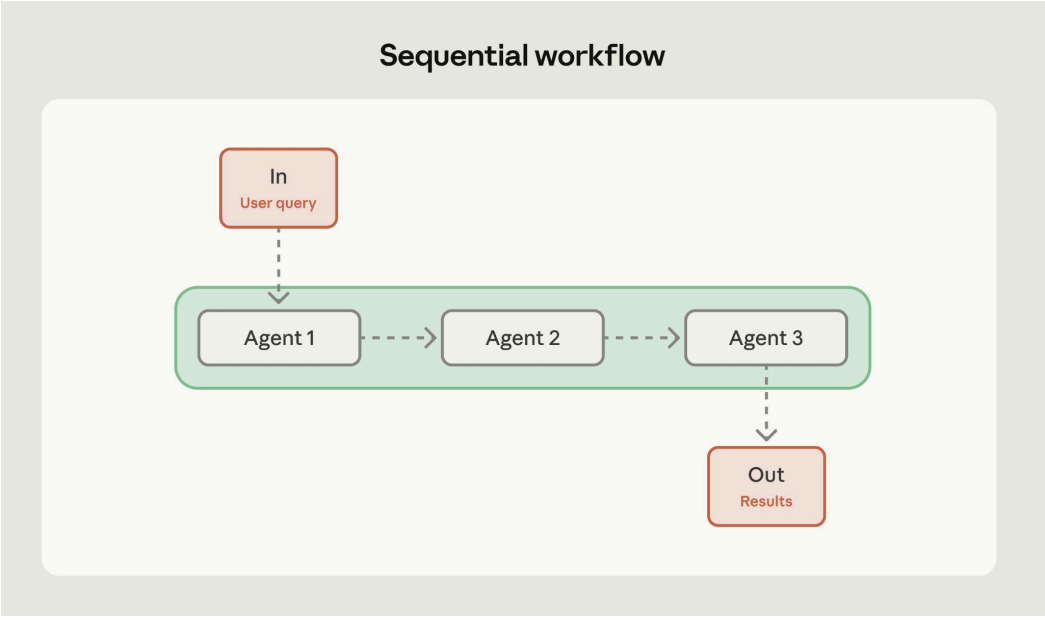
Why use Multi-Agent Systems?

- Task specialisation.
- Parallel execution.
- Better separation of responsibilities.
- Handling complex workflows.
- Improved scalability and modularity.

Common Multi-Agent Coordination Patterns

1 Sequential Pattern

Agents execute tasks one after another.



Example: Content Generation

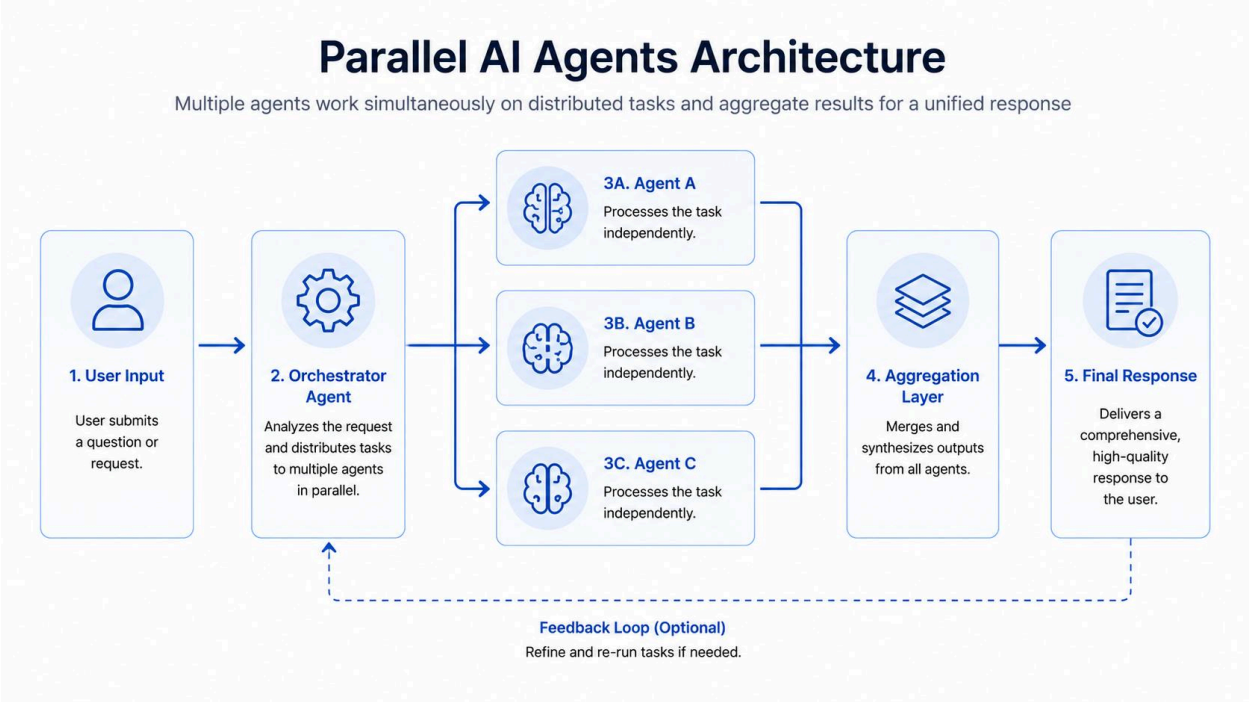
1. Research Agent collects information.
2. Writing Agent creates content.
3. Review Agent checks grammar and accuracy.

Each agent receives the output of the previous agent.

Use case: Document generation, data processing and report creation.

2 Parallel Pattern

Multiple agents execute their tasks simultaneously.



Example: Travel Planning

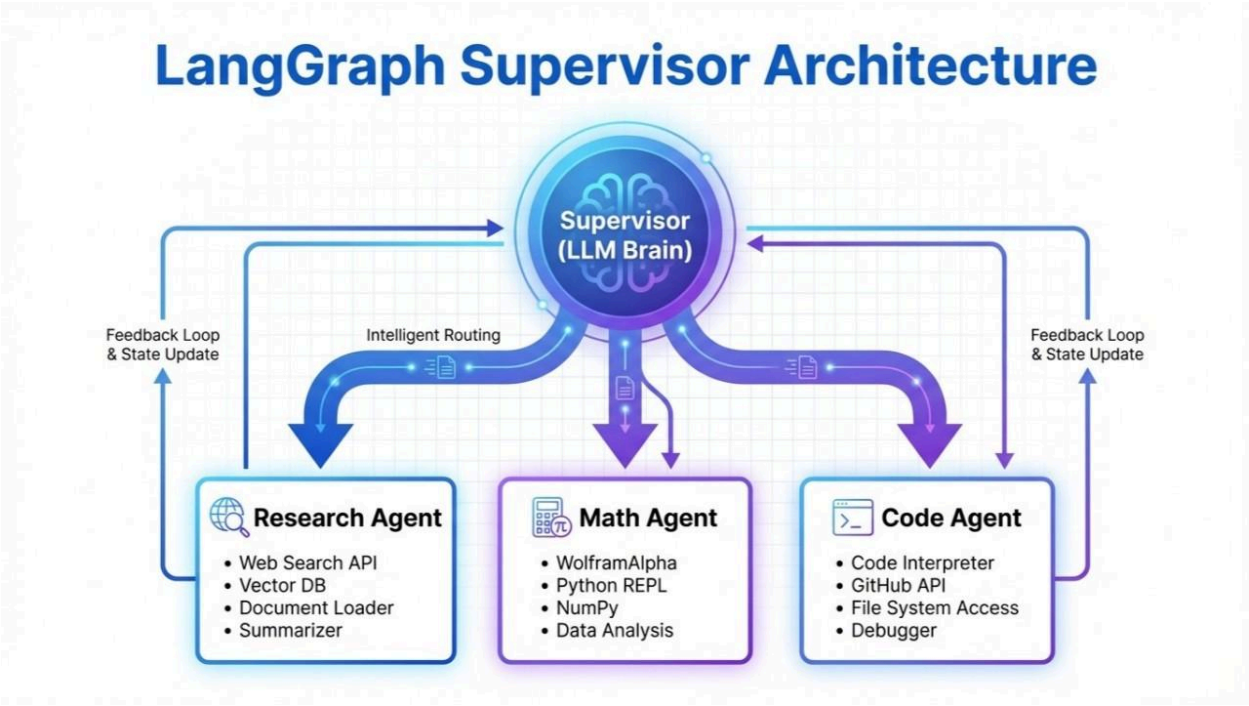
- Flight Agent searches for flights.
- Hotel Agent searches for hotels.
- The Weather Agent checks weather conditions.

A Coordinator Agent combines all results.

Use case: Tasks that can be performed independently.

3 Supervisor Pattern

A Supervisor Agent manages and coordinates multiple specialised agents.



Example: E-commerce Shopping Assistant

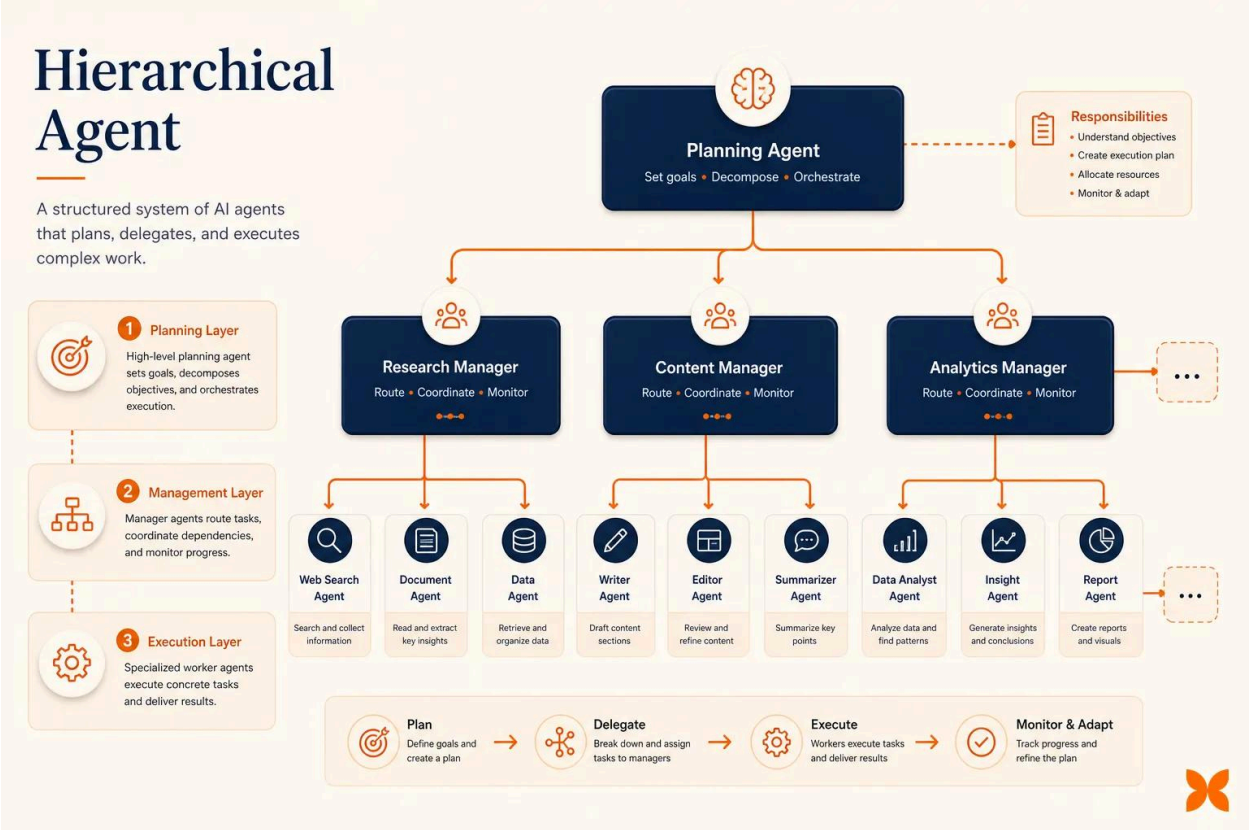
- The user submits a shopping request.
- Supervisor Agent understands the request.
- Product Agent searches for products.
- Price Agent compares prices.
- Review Agent analyses customer reviews.
- The supervisor combines the results and responds.

Use case: Complex tasks requiring centralised control and coordination.

4 Hierarchical Pattern

Agents are organised into multiple levels of responsibility.

A higher-level agent delegates work to lower-level agents, which may further delegate subtasks.



Example: Business Intelligence System

- Main Manager Agent

- Sales Manager Agent
    - Sales Data Agent
    - Revenue Analysis Agent
- Marketing Manager Agent
    - Campaign Analysis Agent
    - Customer Insights Agent

Use case: Large-scale enterprise applications.

5 Handoff Pattern

One agent transfers control to another agent when a task requires different expertise.

Example: Customer Support

1. Customer interacts with a Support Agent.
2. Support Agent identifies a billing issue.
3. It hands over the conversation to a Billing Agent.
4. Billing Agent resolves the issue.

5. The result is communicated to the customer.

Use case: Customer support, healthcare administration and specialised assistance.

6 Collaborative / Debate Pattern

Multiple agents independently analyse the same problem and exchange their findings.

A final agent evaluates their responses and produces a consolidated result.

Example: Research Assistant

- Agent 1 analyses technical feasibility.
- Agent 2 analyses cost.
- Agent 3 identifies risks.
- Evaluator Agent combines the findings.

Use case: Research, decision-making and complex analysis.

MCP

The image mentions Multi-Agent Coordination Patterns (MCP).

One important distinction: In the broader AI ecosystem, MCP commonly stands for Model Context Protocol.

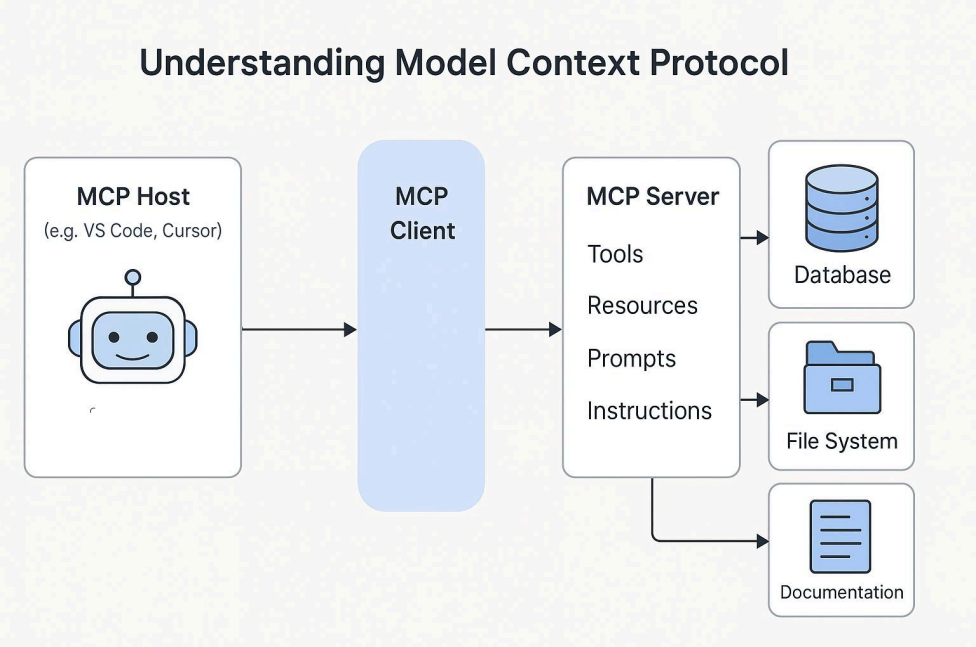
What is Model Context Protocol?

MCP is an open protocol that standardises how AI applications connect to external tools, resources and data sources.

It allows AI applications to interact with external systems through a standardised interface.

Main components:

1. MCP Host: The AI application that uses MCP.
2. MCP Client: Establishes and manages connections to MCP servers.
3. MCP Server: Exposes tools, resources and prompts.
4. Tools: Actions that an AI application can invoke.
5. Resources: Data or contextual information provided to the application.



Example:

An AI assistant connected to:

- A PostgreSQL database MCP server.
- A filesystem MCP server.
- A GitHub MCP server.

The assistant can access information and perform supported operations through these connections, subject to permissions.

Multi-Agent Coordination	Model Context Protocol
Coordinates multiple AI agents	Connects AI applications to external systems
Defines how agents collaborate	Defines how applications communicate with MCP servers
Includes supervisor, parallel and sequential patterns	Includes hosts, clients, servers, tools and resources